Data Structures II
CS201

Spring 2026
March 28, 2026

Project Proposal

Private Local Text Matching using
Single-Layer Navigable Small World (NSW) Graphs

Team Members:

Areeba Izhar ai09625
Arqish Zaria az09714
Noor E Hira nh10085
Shaheer Qureshi sq09647

Instructor: **Faisal Alvi**

1 Abstract

This project builds a simple graph locally in C++ to quickly find similar text phrases by converting each sentence is turned into a number-based vector using Character Trigram Hashing. These vectors become nodes in the graph and Greedy Search is used to find the closest match.

2 Introduction

2.1 Problem Statement

Users often need to quickly find text phrases that are similar, for example:

- Searching personal notes
- Looking through confidential legal documents
- Organizing code snippets

Challenges with existing methods:

- **Arrays or linked lists** requires scanning every entry, slow for large datasets (O(N))
- **Binary Search Trees or Hash Tables** works only for exact matches. For example, a BST or hash table storing "hello world" cannot find "hello worl".
- **Cloud-based solutions** are fast but send data online, which can compromise privacy. For instance, uploading legal documents to an online service could expose the data.

Need:

- A data structure that can **search for similar text efficiently**
- Works **offline**, without sending data to the cloud

2.2 Relevance to Data Structures II

This project is relevant to data structures because it demonstrates how the choice of structure affects efficiency and performance:

- **Graphs:** We use an adjacency list to represent the NSW graph. This goes beyond basic graph traversal to implement "navigability" and "small-world" properties.
- **Greedy Algorithms:** The search algorithm is a greedy best-first search, a fundamental algorithmic paradigm.
- **Priority Queues:** We utilize a Min-Heap to manage candidate nodes during the search process.
- **Hashing:** We use the "Hashing Trick" to convert variable-length text into fixed-size numerical vectors efficiently.

The Navigable Small World (NSW) graph is a probabilistic data structure that builds a network where every node is connected to its nearest neighbors, allowing queries to "hop" through the graph to find matches in logarithmic time.

3 Objectives

The main goals of this project are:

- **Build a Dynamic NSW Graph:** Create a graph that lets us add new text entries without rebuilding the whole structure.
- **Fast Search:** Use a Greedy Search algorithm to quickly find the top-K most similar phrases, much faster than checking every entry one by one.
- **Privacy-Friendly:** Keep all data on the user's computer so nothing is sent to the internet.
- **Convert Text to Vectors:** Turn sentences into numerical vectors using Trigram Hashing, making the system tolerant to typos and small changes.
- **Compare Performance:** Check how fast and accurate our method is compared to a simple linear search, to see if the graph approach is worth it.

4 Methodology

4.1 The Data Structures

The core structure is the **Single-Layer Navigable Small World (NSW) Graph**.

- **Representation:** Adjacency List implemented using C++ Pointers (`std::vector<Node*>`).
- **Node Object:** A struct storing the text, its numerical vector (array), and a list of neighbor pointers.

- **Edges:** Undirected edges implemented as bidirectional pointer references between Node objects. The graph has a maximum degree limit M, which will be tuned between 8 and 16.
- **Entry Point:** A pointer to a designated starting node used to begin every search operation.

4.2 Algorithms

4.2.1 Text Vectorization (Trigram Hashing)

To compare text mathematically, each sentence is converted into a fixed-size vector.

1. **Trigram Extraction:** Split the text into overlapping 3-character sequences (e.g., "hello" → "hel", "ell", "llo").
2. **Feature Hashing:** Each trigram is mapped to an index in a fixed-size vector (dimension 300) using:

$$index = hash(trigram)\%300$$

3. **Vector Construction:** Increment the value at the computed index to build a numerical "fingerprint" of the text.

This approach handles large vocabularies efficiently while using fixed memory.

4.2.2 2. Distance Metric (Cosine Similarity)

We measure the distance between two vectors **A** and **B** using Cosine Similarity.

$$similarity = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

The distance is $1 - similarity$. This metric is robust for text data as it focuses on the angle (direction) rather than the magnitude of the vectors.

4.2.3 3. Insertion Algorithm

When a new sentence is added:

1. Convert text to a vector.
2. Perform a search on the existing graph to find the M closest neighbors.
3. Create bidirectional edges between the new node and these M neighbors.
4. Update the `entry_point` to the newly inserted node to ensure recent additions are easily reachable for future searches.

This ensures the graph remains "navigable" as new nodes are added into the existing network immediately.

4.2.4 4. Greedy Search Algorithm

This is the core logic for finding matches.

1. Start at the `entry_point`.
2. Calculate the distance from the query vector to the current node.
3. Examine all neighbors of the current node.
4. **Greedy Step:** Move to the neighbor that is closest to the query.
5. Repeat until no neighbor is closer than the current node (local minimum reached).
6. **Exploration Factor (`efSearch`):** To make sure the search doesn't get stuck in a poor match, we keep a priority queue (min-heap) of the best nodes found instead of only following the closest neighbor to find better matches.

Algorithm 2
SEARCH-LAYER(q, ep, ef, l_c)
Input: query element q, enter points ep, number of nearest to q elements to return ef, layer number l_c
Output: ef closest neighbors to q

```
v ← ep      // set of visited elements
C ← ep      // set of candidates
W ← ep      // dynamic list of found nearest neighbors
while |C| > 0
  c ← extract nearest element from C to q
  f ← get furthest element from W to q
  if distance(c, q) > distance(f, q)
    break   // all elements in W are evaluated
  for each e ∈ neighbourhood(c) at layer l_c   // update C and W
    if e ∉ v
      v ← v ∪ e
      f ← get furthest element from W to q
      if distance(e, q) < distance(f, q) or |W| < ef
        C ← C ∪ e
        W ← W ∪ e
        if |W| > ef
          remove furthest element from W to q
return W
```

Figure 1: Algorithm 2: SEARCH-LAYER. Sourced from Malkov & Yashunin (2018).

4.3 Tools and Technologies

- **Language:** C++ 17 (Standard Library only).
- **Data Structures:** `std::vector` (for adjacency lists), `std::priority_queue` (for heaps), `std::unordered_set` (for visited tracking).
- **Memory Management:** Nodes are dynamically allocated. A master list of Node pointers is maintained to ensure proper cleanup and memory safety.
- **Privacy:** All processing is local (no network calls). Data persistence handled via standard file I/O.
- **Testing:** Custom test cases comparing Brute-Force vs. NSW runtimes.

4.4 Expected Challenges

- **Getting Stuck in Poor Matches:** Sometimes the search might stop at a node that looks good locally, but isn't the best overall. We plan to fix this by adjusting `efSearch`, which keeps a larger set of candidate nodes during the search.
- **Choosing Graph Parameters:** Setting the right maximum number of neighbors (M) is important. Too few neighbors can make the graph disconnected, while too many can slow down the search. We will test values between 8 and 16.
- **Measuring Accuracy:** To make sure the search works well, we will check our **Hit Rate**, which tells us how many of the true top matches the algorithm actually finds.

5 Expected Outcomes

By the end of this project, we will deliver:

1. A fully functional CLI tool for adding and searching private text notes.
2. A working implementation of the NSW Graph that demonstrates faster search times than brute-force as the dataset grows (demonstrating $O(\log N)$ behavior vs $O(N)$).
3. A robust understanding of Proximity Graphs, able to be explained during the Viva using hand-drawn diagrams of the "greedy walk."

6 References

1. Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

2. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (For Vector Space Models and Cosine Similarity).